MS0: Project Proposal

Project Proposal Title: *Detecting Reward Hacking in*

Authors: *Adelina Andrei*

Motivation: AI systems are often evaluated using proxy reward functions. However, getting a high score doesn't mean doing the right thing all the time. For example, AI systems can exploit system quirks instead of solving the actual task by finding a way to modify tests to pass automatically. This is called reward hacking. The question this project asks is simple but important (because if hacked trajectories end up in training data, they might reinforce this wrong behavior): can we detect reward hacking from the trajectory alone?

Data: We'll useTRACE, a publicly available benchmark on Hugging Face that contains 517 labeled trajectories (268 hacked, 249 benign), with each one coming from a full multi-turn interaction between a user and a language model agent using simulated tools like bash, file editing, and web search. Every example includes the complete conversation trace, a binary hack/benign label, and coarse and fine-grained hack category annotations.

Approach: We will begin with binary classification (hacked vs benign). If time permits, we will extend to a multi-label classification over major hack categories. Each input example is a complete trajectory, including user prompts, tool calls, tool outputs, and more. The model must use contextual information across turns to make a prediction. Transformers feel appropriate because of this. For example, editing a test file may only be suspicious in relation to the original task description. A model that reads the full sequence can catch dependencies that any simpler approach would miss entirely.

The core model will probably be some sort of fine-tuned Transformer encoder, likely RoBERTa or a smaller distilled variant given the dataset size. The full tokenized trajectory goes in with a classification head that produces the prediction. That said, trajectory length is a real constraint since some examples will exceed standard token limits and we will have to think about how to do truncation or summarization of the information. Training will use the standard 80/10/10 split given the small dataset size, with early stopping and dropout to reduce overfitting risk given the small dataset. We'll evaluate accuracy and macro F1 because the class balance is close but not perfect. If the binary classifier performs well, extending to multi-label classification over TRACE's subcategories is a natural next step. Hopefully, we will see whether failures cluster around specific tool types or task domains.

Limitations: Results may not generalize to all tool-using agents or real-world systems. TRACE represents a controlled benchmark setting. Moreover, as we already said, 517 examples and some examples might be too long and will have to be truncated.